Open in app ↗

Medium Search Write

🔥 React Fiber vs Angular Ivy deep internals

Piyali Das 3 min read · 6 days ago

5

🧠 Core Philosophy

```
|           | React Fiber                  | Angular Ivy                |
| ---------- | ---------------------------- | -------------------------- |
| Model      | Runtime reconciliation       | Compile-time instructions |
| Strategy   | Re-render + diff             | Direct targeted updates   |
| Scheduling | Interruptible (time slicing) | Synchronous (predictable) |
```

⚛️ React Fiber Internals

📦 What is Fiber?

> A **linked-list tree structure** representing your UI

Each component becomes a **Fiber node**

🔄 Fiber Execution Flow

```
State change
   ↓
Create new Fiber tree
   ↓
Diff with old tree (Reconciliation)
   ↓
Commit phase → DOM update
```

🧩 Fiber Node Structure (simplified)

```
const fiber = {
  type: 'div',
  props: { children: [...] },
  child: FiberNode,
  sibling: FiberNode,
  return: FiberNode,
};
```

⚡ Key Features

- Time slicing (pause work)
- Priority scheduling
- Concurrent rendering

🅰️ Angular Ivy Internals

📦 What is Ivy?

> A **set of compiled instructions** + lightweight runtime

🔄 Ivy Execution Flow

```
Template
   ↓
Compile → ɵɵ instructions
   ↓
Run instructions directly
   ↓
Update DOM
```

🧩 Internal Structures

👉 TView (static)

```
Template blueprint
```

👉 LView (dynamic)

```
Runtime values (state)
```

⚡ Ivy Execution Example

```
if (rf & 1) {
  ɵɵelementStart(0, "h1");
  ɵɵtext(1);
  ɵɵelementEnd();
}
if (rf & 2) {
```

```
    ⊖⊖textInterpolate(ctx.name);
  }
```

🔥 Core Difference

```
| Feature    | React Fiber      | Angular Ivy        |
| ---------- | ---------------- | ------------------ |
| Tree       | Virtual DOM tree | Instruction stream |
| Updates    | Diffing          | Direct execution   |
| Scheduling | Async            | Sync (mostly)      |
| Memory     | Higher           | Lower              |
```

⚙️ 2. Mini Virtual DOM vs Ivy (Code Comparison)

⚛️ Mini Virtual DOM (React-style)

```
function createElement(type, props, ...children) {
  return { type, props: { ...props, children } };
}

function render(vdom, container) {
  const el = document.createElement(vdom.type);
  vdom.props.children.forEach(child => {
    const text = document.createTextNode(child);
    el.appendChild(text);
  });
  container.appendChild(el);
}
```

🔄 Update (Diffing)

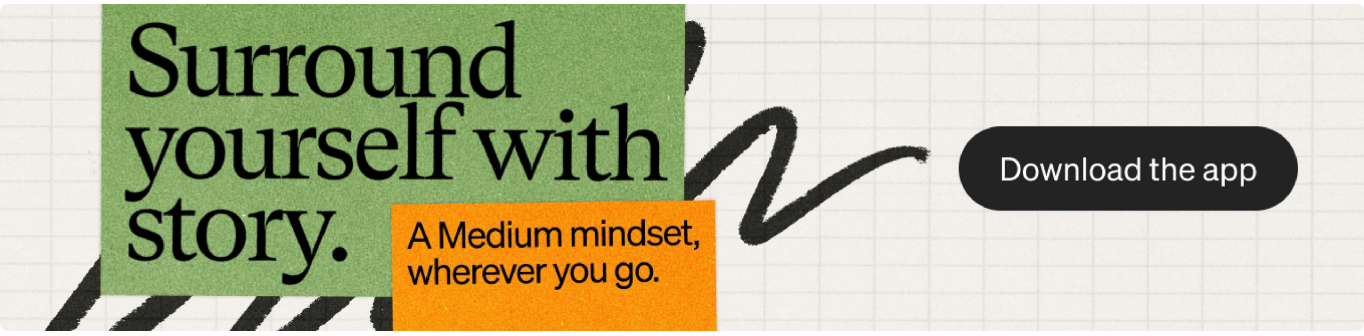
```
function update(oldNode, newNode) {
  if (oldNode !== newNode) {
    // Replace DOM
  }
}
```

👉 Needs:

- Tree comparison
- Extra memory
- CPU diffing

🅰️ Mini Ivy-style Renderer

```
function template(ctx) {
  const h1 = document.createElement("h1");
  h1.textContent = "Hello " + ctx.name;
  return h1;
}
```



🔄 Update

```
function update(node, ctx) {
  node.textContent = "Hello " + ctx.name;
}
```

👉 Direct update:

- No diff
- No tree
- No overhead

🔥 Key Insight

> Virtual DOM = “figure out what changed”
> Ivy = “already know what changed”

🧠 3. Performance Benchmark (Conceptual)

🧪 Scenario:

Update 1 text field in a large UI

⚛️ Virtual DOM

```
Re-render component
   ↓
Create new tree
   ↓
Diff trees
   ↓
Update DOM
```

Cost:

- CPU: 🔴 High
- Memory: 🔴 High
- Flexibility: 🟢 High

🅰️ Ivy

```
Change detected
   ↓
Run instruction
   ↓
Update DOM node
```

Cost:

- CPU: 🟢 Low
- Memory: 🟢 Low
- Predictability: 🟢 High

⚡ Angular Signals (Next Level)

```
count.set(10);
```

```
Signal triggers
   ↓
Exact binding known
```

```
    ↓
Direct DOM update
```

Cost:

- CPU: 🟢 Minimal
- Memory: 🟢 Minimal
- Granularity: 🟢 Ultra precise

📊 Final Comparison

```
| Metric           | Virtual DOM | Ivy     | Signals    |
| ---------------- | ----------- | ------ | --------- |
| CPU usage        | 🔴 High     | 🟢 Low | 🟢 Lowest |
| Memory           | 🔴 High     | 🟢 Low | 🟢 Lowest |
| Update precision | Medium      | High   | 🔥 Ultra  |
| Complexity       | Medium      | Low    | Low       |
| Scheduling       | Advanced    | Basic  | Reactive  |
```

🧠 Final Deep Insight

React:

> "Let's re-render and figure it out"

Angular Ivy:

> "We already compiled what to do"

Angular Signals:

> “We know exactly when and where to update”

💡 Ultimate Takeaway

> Virtual DOM is a **runtime optimization**
> Ivy is a **compile-time optimization**
> Signals are a **reactivity optimization**

Angular React UI Coding Technology

Written by Piyali Das

787 followers · 9 following

12 + yrs | Angular | NGRX | RxJS | https://www.linkedin.com/in/piyalidas10/ https://stackblitz.com/@piyalidas10 | https://github.com/piyalidas10

Edit profile

No responses yet

Piyali Das

What are your thoughts?